Subject : LP-3

Section : DAA

Experiment : DAA-1

Fibonacci Numbers using Recursive and Non-Recursive Methods

Code :

```
def fibonacci_iterative(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b

def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

n = 7
print("Non-recursive Fibonacci:")
fibonacci_iterative(n)

print("\\nRecursive Fibonacci:")
for i in range(n):
    print(fibonacci_recursive(i), end=" ")

print("\\n\\nTime Complexity:")
print("Iterative: O(n)")
print("Recursive: O(2^n)")
print("Space Complexity:")
print("Iterative: O(1) auxiliary space")
print("Recursive: O(n) recursion stack")
```

Output :

```
Non-recursive Fibonacci:
0 1 1 2 3 5 8

Recursive Fibonacci:
0 1 1 2 3 5 8

Time Complexity:
Iterative: O(n)
Recursive: O(2^n)
Space Complexity:
Iterative: O(1) auxiliary space
Recursive: O(n) recursion stack
```

Note: The recursive method has exponential time complexity because it repeats calculations.

Subject : LP-3

Section : DAA

Experiment : DAA-2

Huffman Encoding using Greedy Method

Code :

```
import heapq
from collections import Counter

def huffman(text):
    heap = [[freq, [[char, ""]]]
            for char, freq in Counter(text).items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        for pair in left[1]:
            pair[1] = "0" + pair[1]
        for pair in right[1]:
            pair[1] = "1" + pair[1]

        heapq.heappush(
            heap, [left[0] + right[0], left[1] + right[1]]
        )

    return sorted(heap[0][1], key=lambda x: x[0])

text = "aabbc"
print("Character Frequencies:", dict(Counter(text)))
print("Huffman Codes:")
for char, code in huffman(text):
    print(char, ":", code)
```

Output :

```
Character Frequencies: {'a': 2, 'b': 2, 'c': 1}
Huffman Codes:
a : 11
b : 0
c : 10
```

Note: Huffman codes can differ when frequencies tie; the generated codes remain valid prefix codes.

Subject : LP-3

Section : DAA

Experiment : DAA-3

Fractional Knapsack using Greedy Method

Code :

```
def fractional_knapsack(capacity, items):
    # Each item is (value, weight)
    items = sorted(items, key=lambda x: x[0] / x[1], reverse=True)
    total_value = 0

    for value, weight in items:
        take = min(capacity, weight)
        total_value += take * value / weight
        capacity -= take
        print(f"Taken weight: {take}, added value: {take * value / weight:g}")
        if capacity == 0:
            break

    return total_value

items = [(60, 10), (100, 20), (120, 30)]
capacity = 50
print("Maximum value:", fractional_knapsack(capacity, items))
```

Output :

```
Taken weight: 10, added value: 60
Taken weight: 20, added value: 100
Taken weight: 20, added value: 80
Maximum value: 240.0
```

Note: Items are represented as (value, weight). The final item is taken fractionally.

Subject : LP-3

Section : DAA

Experiment : DAA-4

0/1 Knapsack using Dynamic Programming

Code :

```
def knapsack(capacity, weights, values):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i - 1][w]
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    dp[i][w],
                    values[i - 1] + dp[i - 1][w - weights[i - 1]]
                )

    return dp[n][capacity]

weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50

print("Maximum value:", knapsack(capacity, weights, values))
print("Time Complexity: O(nW)")
print("Space Complexity: O(nW)")
```

Output :

```
Maximum value: 220
Time Complexity: O(nW)
Space Complexity: O(nW)
```

Note: For capacity 50, the optimal selection is weights 20 and 30, with total value 220.